



CMPT 295

Unit – Microprocessor Design & Instruction Execution

Lecture 28 – Intro to Microprocessor Design

Last Lecture

- ✓ We created our own **instruction set (IS)**
 - ✓ **x295M**: “Memory only”
- ✓ We checked whether or not **x295M** followed **ISA** design guidelines?
- ✓ How good was our **x295M**? -> Evaluation of **ISA**
 - ✓ We compared **x295M** with **MIPS**
 - ✓ Compiled a sample **C** program (code) using both **ISAs**
 - ✓ Then evaluated both **ISAs**
- ✓ When we evaluate **ISA** we use the **Memory Traffic** criteria:
 - ✓ This means that we examine the effect of the **von Neumann bottleneck** on the execution time of our program by counting **number of memory accesses**
 - ✓ The fewer memory accesses our program makes, the faster it executes, hence the “better” it is
 - ✓ Hence, reducing the **number of memory accesses** decreases the effect of **von Neumann bottleneck** on our program execution

Today's Menu

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design guidelines
 - Look at an example of an instruction set: MIPS
 - Create our own
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Staged execution of machine instructions
 - Pipelined execution of machine instructions + Hazard

The Big Picture

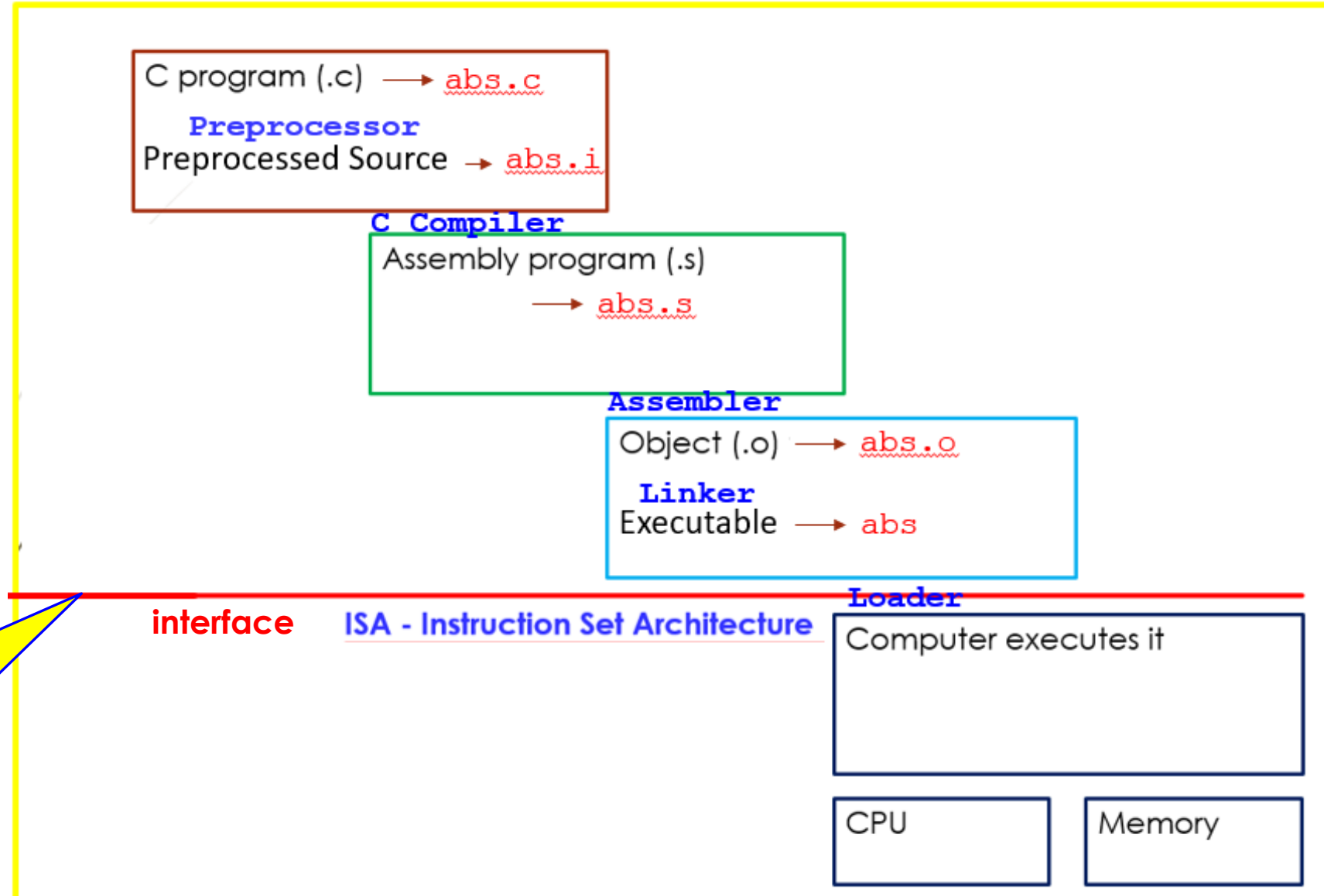
Now that we have had a look at instruction set architectures (**ISA**) and their design:

1. Memory model
2. Registers
3. Instruction set
 - Assembly instructions
 - Machine code instructions:

0x00400000

```
100011 11101 10001 0000 0000 0000 0000
100011 11101 10010 0000 0000 0000 0100
000000 10001 10010 10011 00000 100000
000000 10001 10010 10100 00000 100010
000000 10011 10100 10101 00000 100100
101011 10101 11101 0000 0000 0000 0008
```

... let's step over this interface and explore how the microprocessor is constructed so that it can execute these machine code instructions



Example: Datapath of a **MIPS** microprocessor

Here is a peek at how **microprocessor** hardware works when the PC is updated as part of the microprocessor *fetch-decode-execute* loop:

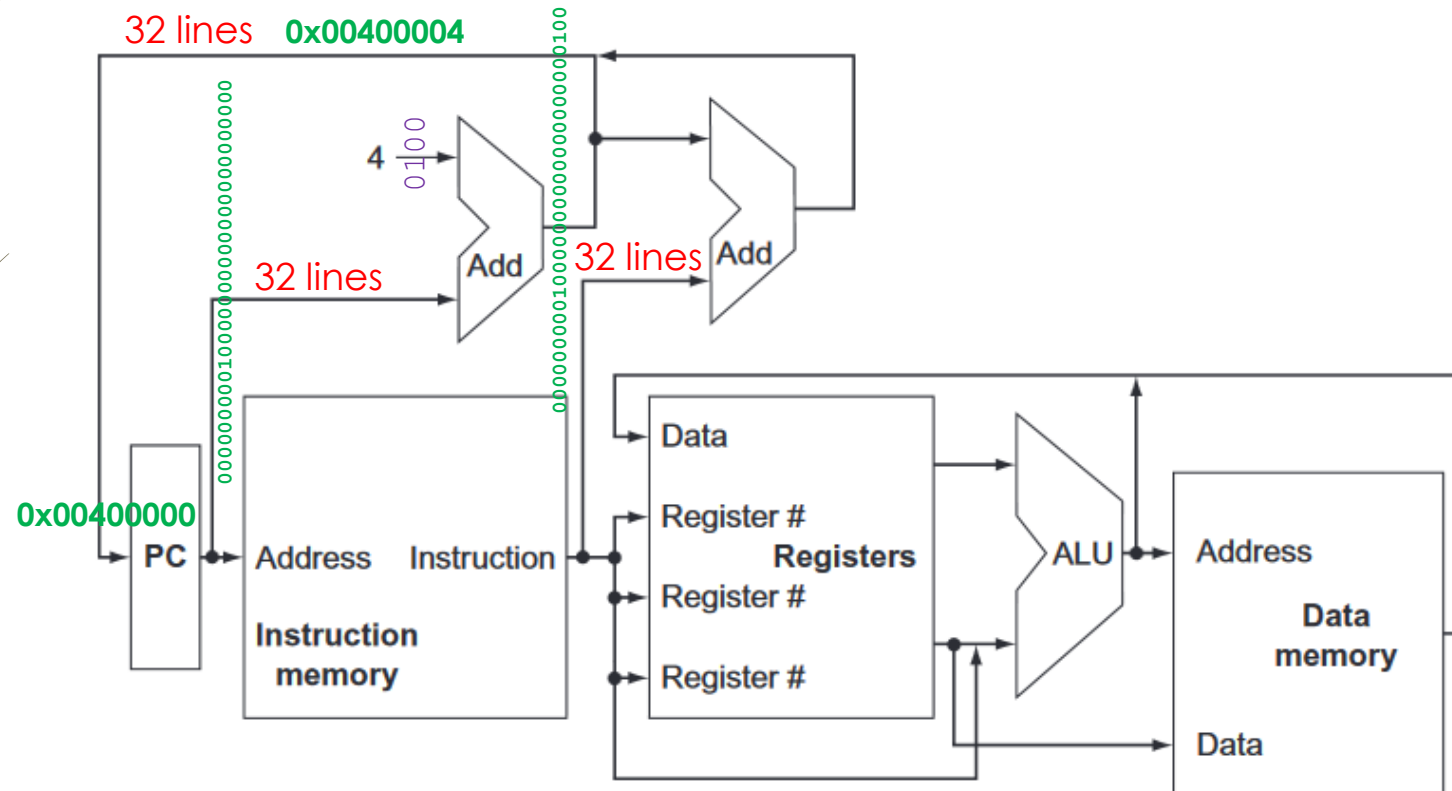


FIGURE 4.1 An abstract view of the implementation of the MIPS subset showing the major functional units and the major connections between them.

Source: Page 246 in Patterson and Hennessy

Example: Datapath of a **MIPS** microprocessor

- The details

- Consider the memory address **0x00400000**, i.e., **00000000010000000000000000000000** which holds the **MIPS** machine instruction **100011 11101 10001 0000000000000000** (1_w from Lecture 27)
- This value **0x00400000** is contained in the PC and represents 32 0's and 1's i.e., signals and these signals are traveling over 32 input lines to the ADDER
- **4** i.e., **0100₂** are the signals (0's and 1's) traveling on the other input lines of the ADDER
- The hardware of the ADDER adds both inputs and produces **0x00400004** i.e., the memory address where the microprocessor will find the next **MIPS** machine code instruction **100011 11101 10010 0000000000000100** (1_w from Lecture 27)
- The signals **0x00400004** travel over the 32 input lines to the PC and overwrite its current value: **0x00400000**

Digital circuits

- In order to understand **how** the **microprocessor** executes these **machine code instructions** (series of 0's and 1's), we need to have a look at the components of a microprocessor and how they function:

Types of components found in a digital system such as a microprocessor

1. **Combinational logic circuit** -> compute functions e.g., ADD
2. **Sequential logic circuit (storage/memory)** -> store bits
3. **Clock** -> its signals regulate the update of sequential logic circuit

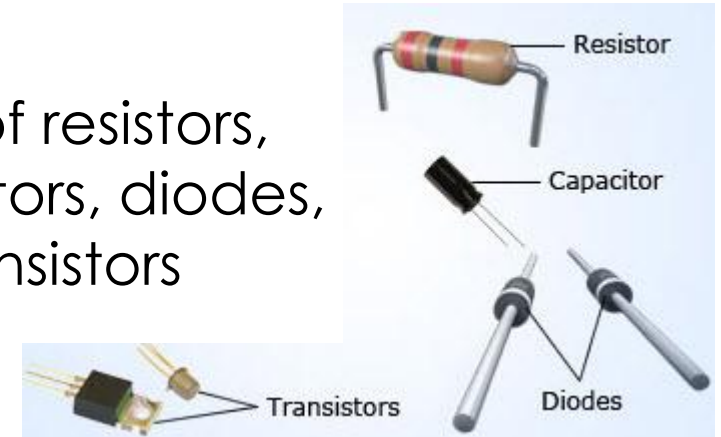
and what affects the execution speed of these components such as **propagation delay**

- So, we need to understand a few things about **digital circuits**

Microprocessor



- Made of resistors, capacitors, diodes, and transistors



Source: <https://www.elprocus.com/semiconductor-devices-types-and-applications>

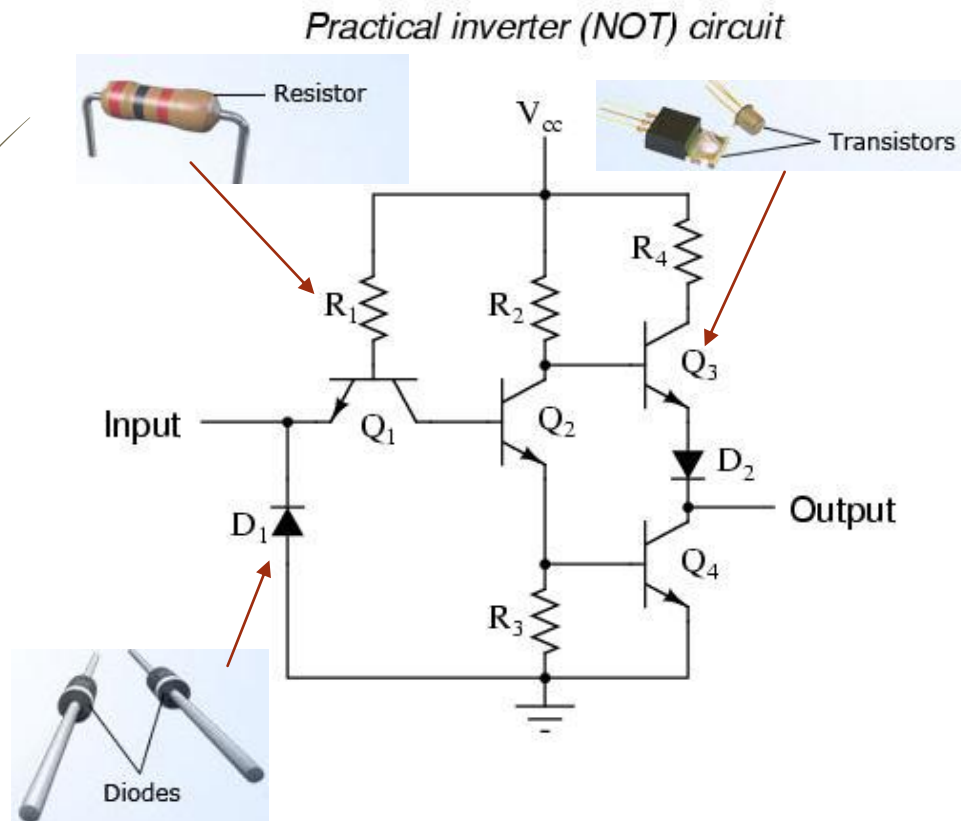
- For example, 10-core Core i7 Broadwell-E (2016) from Intel contains 3,200,000,000 transistors

Source: https://en.wikipedia.org/wiki/Transistor_count

- Lots of incredibly small components
- They are abstracted into **logic gates**

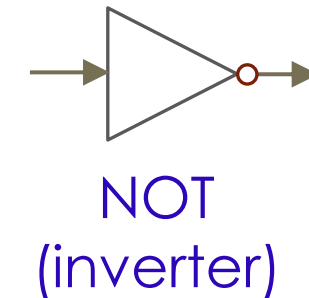
Example of a logic gate

Here is how a **NOT** logic gate is **constructed**:



Source: http://www.cybermike.net/reference/liec_book/Digital/DIGI_3.html

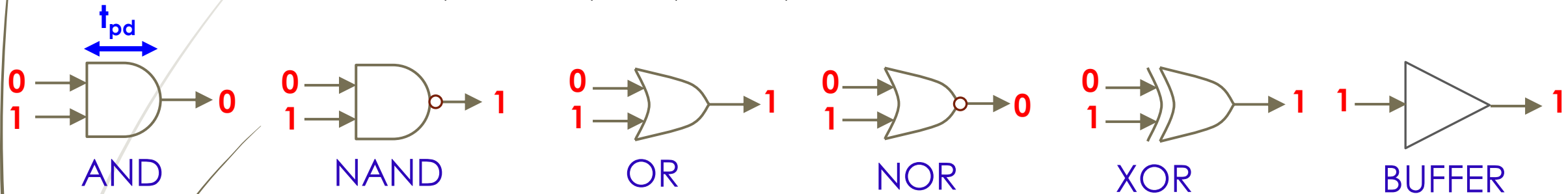
Here is how a **NOT** logic gate is **abstracted**:



Instead of drawing logic gates using their electronic components (on the left), we hide these components using a **black box** approach, i.e., representing a logic gate using a simplified diagram

Logic gates

- Electronic devices that perform a Boolean function such as AND, NAND, OR, XOR, NOT

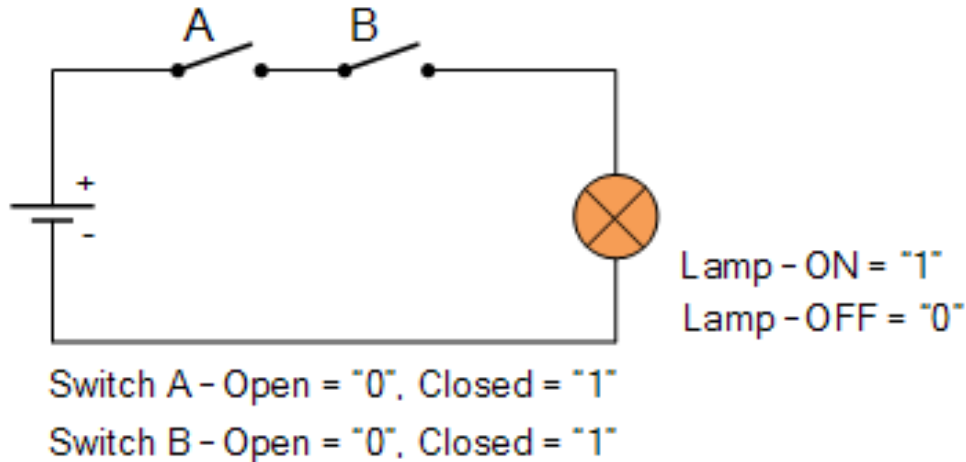


- **Input:** a signal i.e., 0 or 1 (abstraction of voltage levels) travels along the input line
- **Output:** After a time delay (**propagation delay t_{pd}**), a signal, i.e., 0 or 1 travels along the output line
- **Always active**
 - As soon as signal (0 or 1) travels along the input lines, the logical gate produces a result, i.e., a signal (0 or 1) which then travels along the output line

Behaviour of logic gates

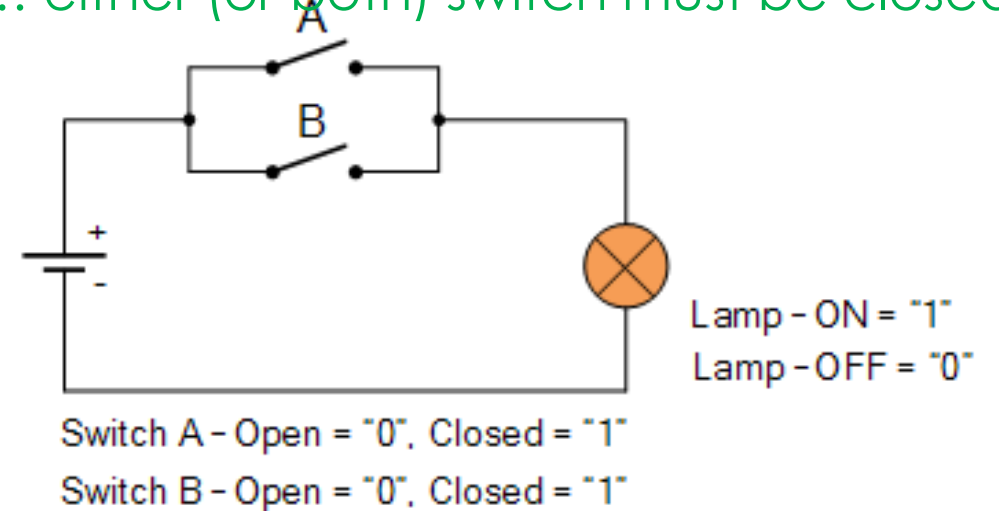
- Here is a diagram representing the behaviour of an AND logic gate:

... both switches must be closed!



- Here is a diagram representing the behaviour of an OR logic gate:

... either (or both) switch must be closed!



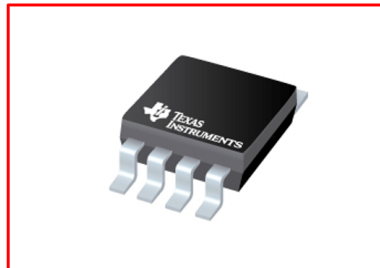
Propagation delay t_{pd}

- Definition: Longest time elapsed between the application of a signal to an input line and the occurrence of the corresponding result i.e., signal on the output line
- t_{pd} often expressed in picosecond (10^{-12} seconds) to nanosecond (10^{-9} seconds)

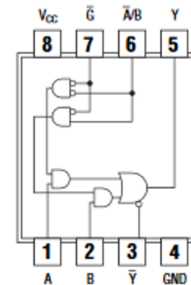
1. Combinational logic circuits

- **Definition:** Computes a function, where its output is based only on its input
- Made by connecting several logic gates together
- Compute more complex functions than just AND or XOR

Example of an integrated circuit (IC)
SN74LVC2G157

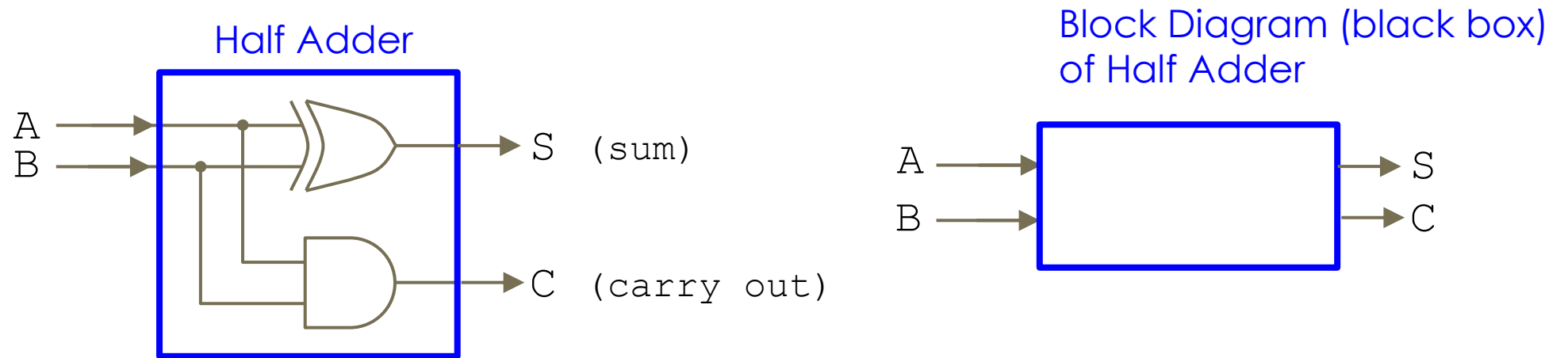


2G157
SINGLE 2-LINE TO 1-LINE DATA
SELECTOR/MULTIPLEXER



1. Combinational logic circuits

- **Example:** Combinational logic circuit that adds 2 bits together



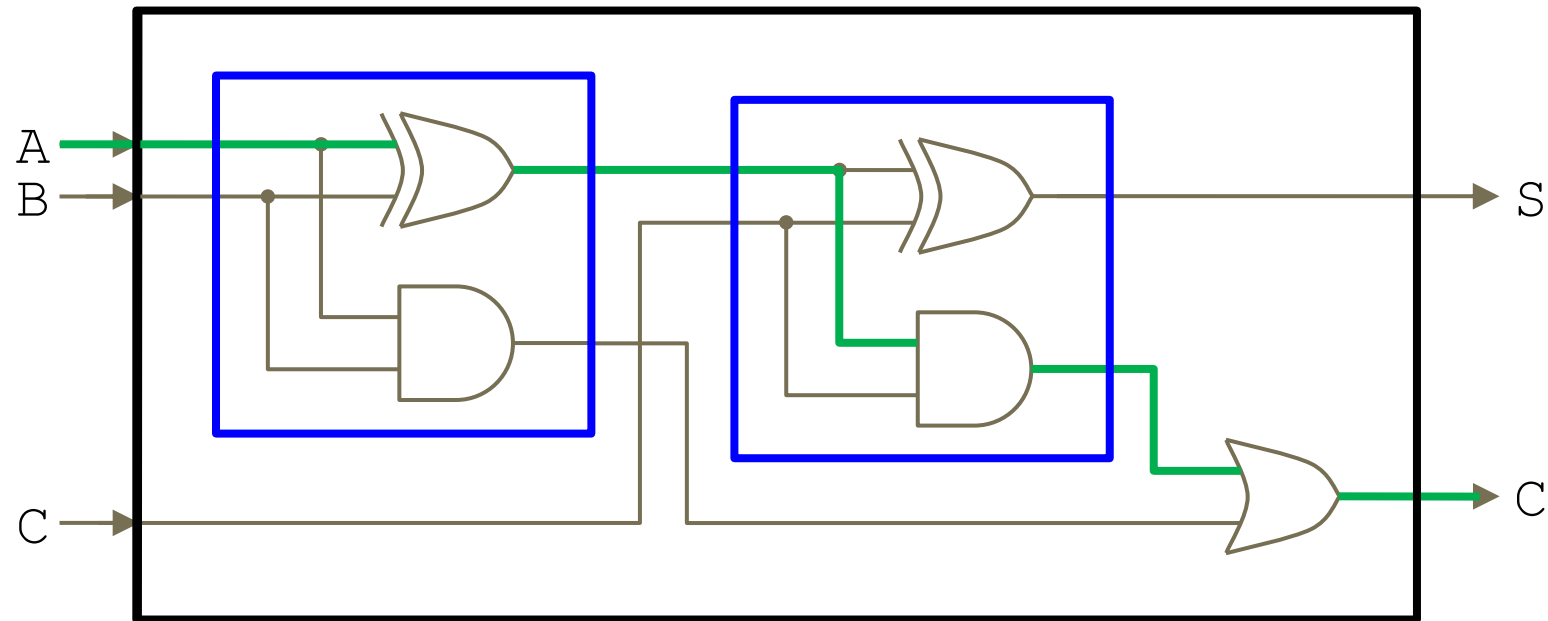
- Propagation delay: "1 gate" delay
- <http://sullystationtechnologies.com/ichalfadder.html>

More examples in our Textbook

- Combinational circuit to test for bit equality -> Figure 4.10
- Combinational circuit to test for word equality -> Figure 4.12
- Single-bit multiplexor circuit -> Figure 4.11
- Word-level multiplexor circuit -> Figure 4.13

1. Combinational logic circuit

- Can connect several combinational logic circuits together to perform more complex functions and/or to perform function on a wider input
- What does this circuit do?

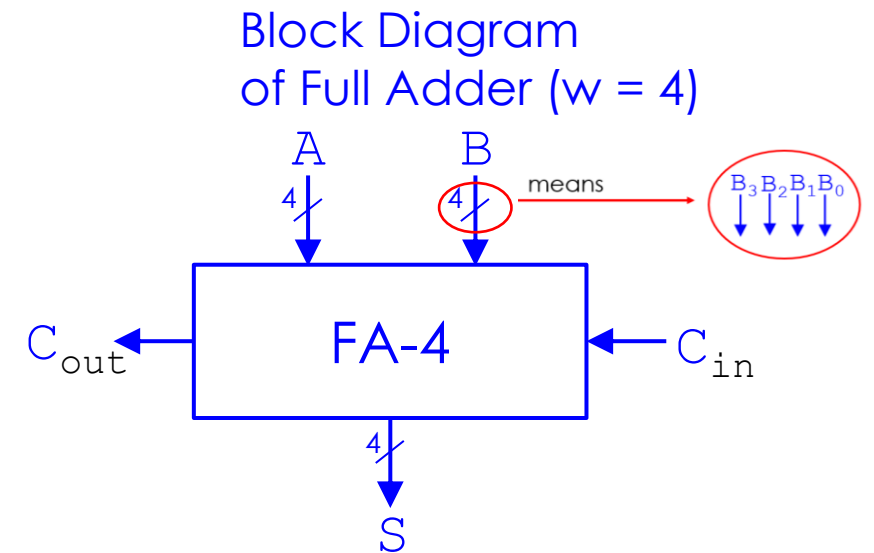
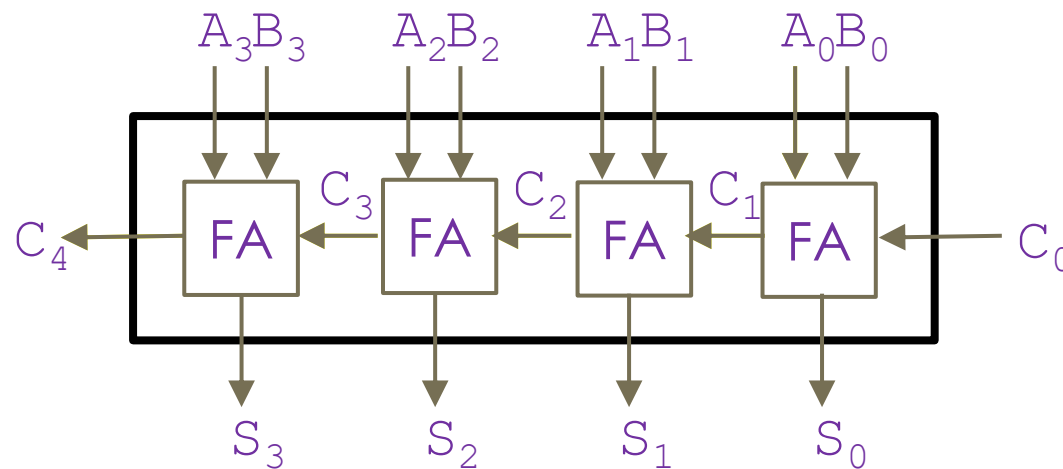


- Propagation delay: "3 gate" delay

1. Combinational logic circuit

- We can connect several Full Adders together so that the resulting circuit can operate on word size data such as integral values and memory addresses
- Example: Full Adder operating on data of word size 4 ($w = 4$)

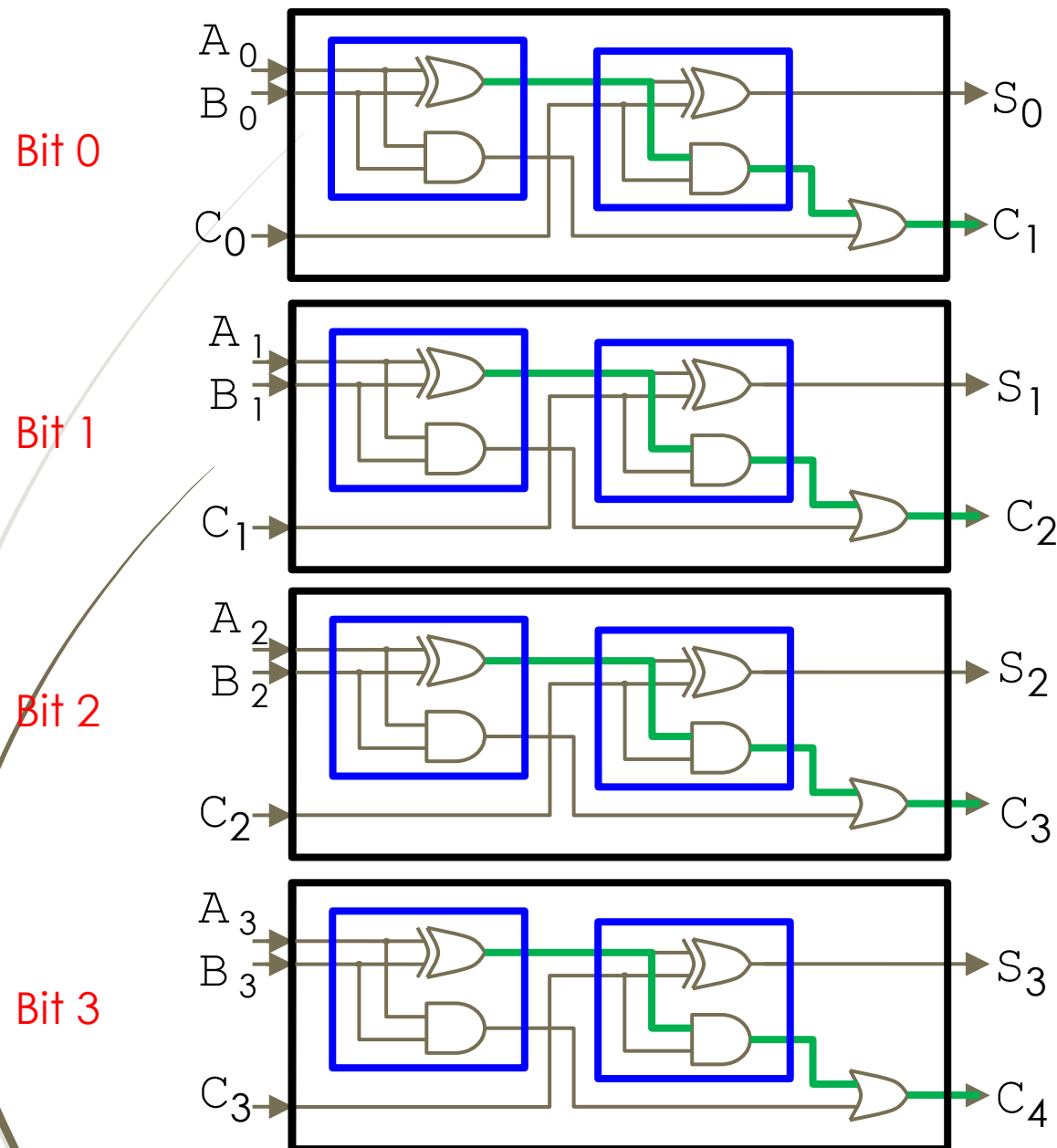
aka 4-bit ripple carry adder



- Propagation delay?

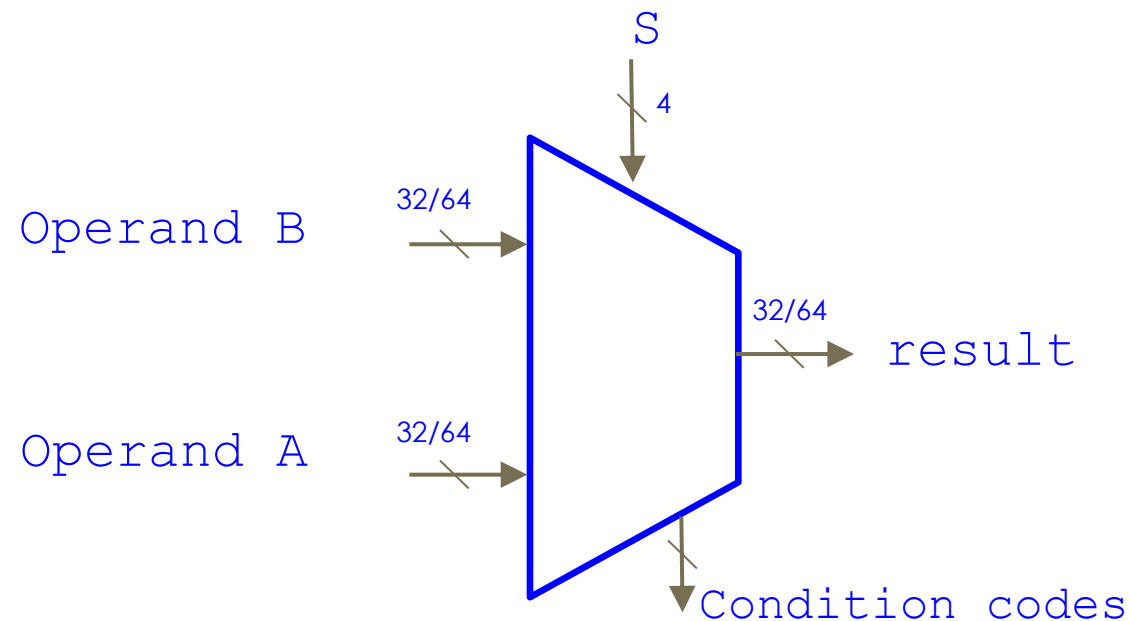
Challenge

- Why is the propagation delay of this Full Adder 4 (also known as a 4-bit (w-bit) ripple carry adder) 9 gates ($2w + 1$ gates) where w is the number of bits (as opposed to 12 gates $\rightarrow 4 \times 3$ gate p.d.)?



ALU - Example of combinational logic circuit

- Arithmetic/logic unit (ALU) found in microprocessor
 - Black box diagram
- ALU executes *data manipulation machine instructions* (defined in the ISA)



S	operation
0001	A + B
0010	A - B
1000	A * B
0100	A ^ B
0101	A & B

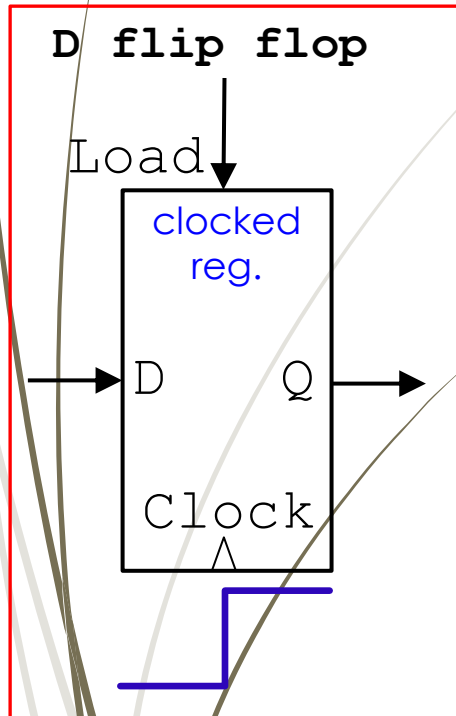
1. Combinational logic circuit - Summary

- Computes a function, where its outputs is based only on its inputs
- Made by connecting several logic gates together
- Acyclic: there are no feedback loops – output cannot be its input
- Does not require a clock signal to activate it
- Does not store (remember) its output

2. Sequential logic circuit (storage/memory)

- **Definition:** Stores (remembers) bits
- Contains specific type of logic gates called “flip-flop”
 - Requires a clock signal and a load signal to activate it
 - 1 flip flop remembers 1 bit
- Memory is composed of a very large number of flip flops
 - **Memory elements**
- Example of sequential logic circuits:
 1. RAM (see video)
 2. Register file -> register *id* used as *address*

3. Clock



- System-wide clock
- **Definition:** Clock “tick” updates all memory elements
 - A system-wide clock sends pulses: 0 1 0 1 0 1 0 1
 - Clock period: 1 full cycle duration: “tick” = rising edge of clock
 - Clock frequency: 1/period
- How does D flip flop works:
 - Has **current state** (0 or 1) which it outputs on output line **Q**
 - Input line **D** (0 or 1) and **Load** line that activates it
 - When clock “ticks” and the value 1 is on Load line, the value on the input line come into the flip flop (next state). Then the D flip flop remembers the value until the clock “ticks” again with 1 on the Load line.

Function Table

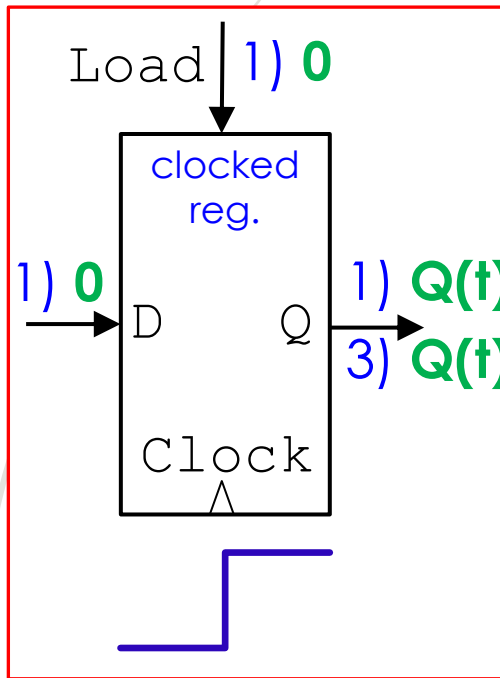
Load	D	$Q(t + 1)$
0	X	$Q(t)$
1	0	0
1	1	1

Imagine there is a door at the entrance of the D flip flop that briefly opens whenever the clock “ticks”.

Function Table

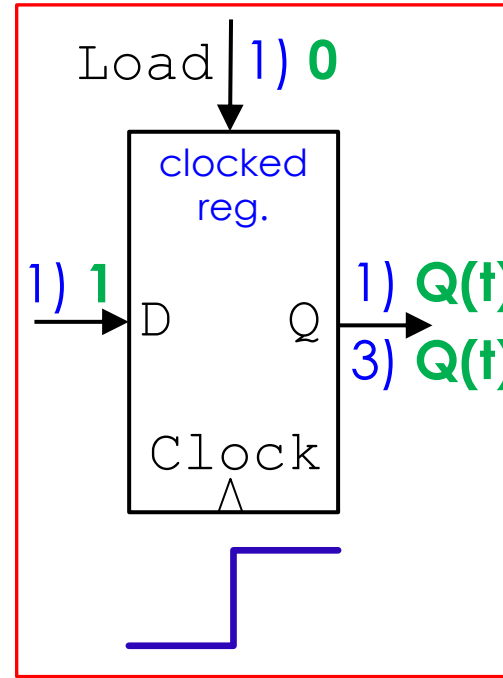
Load	D	$Q(t + 1) \rightarrow$ next state
0	X	$Q(t) \rightarrow$ current state
1	0	0
1	1	1

How a Clocked Register function!



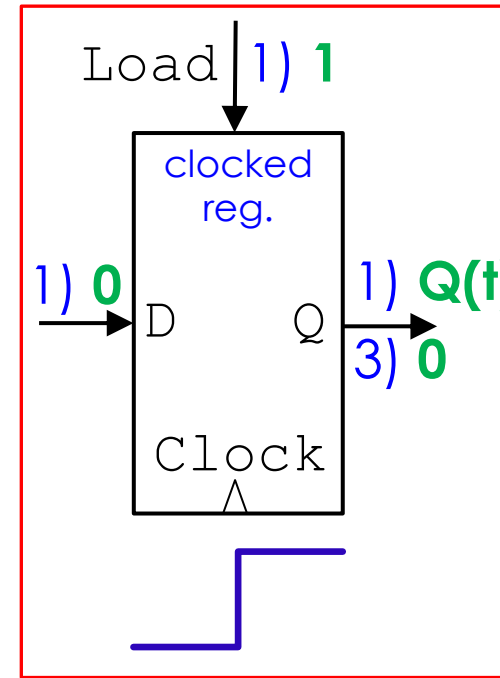
2) When the clock "ticks", i.e., when edge of clock rises

Row 1 of Function Table



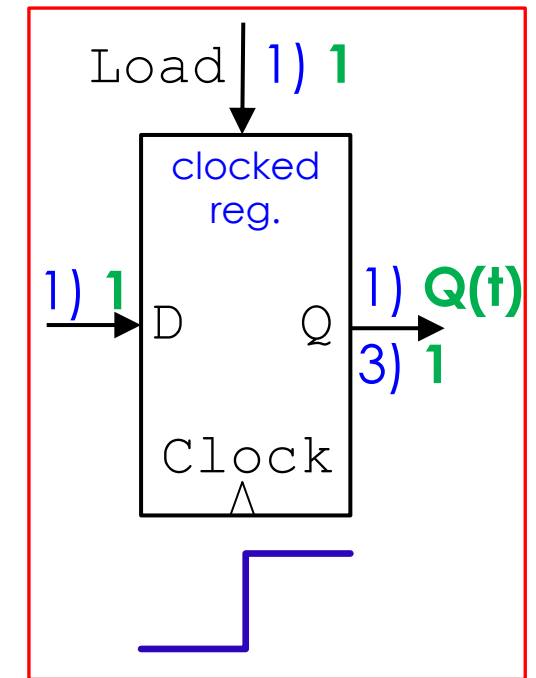
2) When the clock "ticks", i.e., when edge of clock rises

Row 1 of Function Table



2) When the clock "ticks", i.e., when edge of clock rises

Row 2 of Function Table

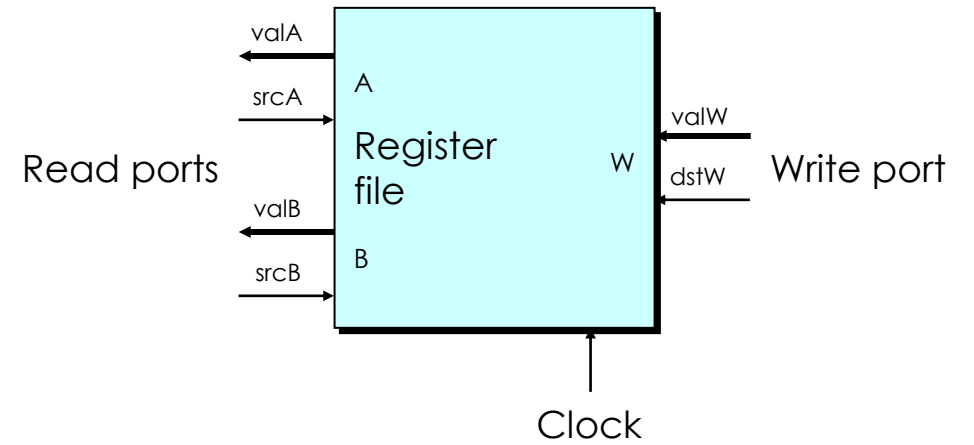


2) When the clock "ticks", i.e., when edge of clock rises

Row 3 of Function Table

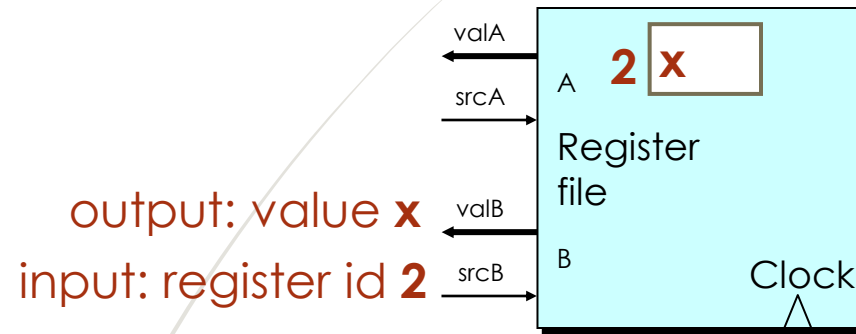
Let's have a look at an example of a sequential logic circuit (memory element): **registers!**

Register File



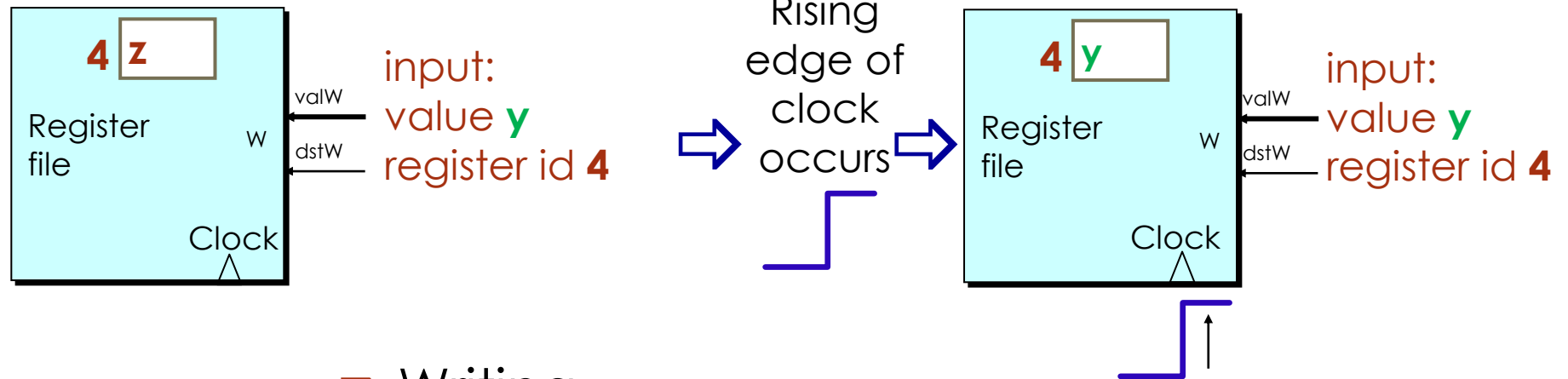
- Stores multiple words
- Each word is the value held in a **program register**
 - `%rax`, `%rsp`, etc.
 - **Register id** serves as *address* to specify which word to read or write
- Multiple ports
 - What is a **port**?
 - Can read and/or write multiple words in one clock cycle
 - Each port has a separate address line and a separate data input or data output line

Register file – how it works?



➤ Reading

- *Combinational logic circuit* – does not need clock
- Output data generated based on input address only, after some propagation delay

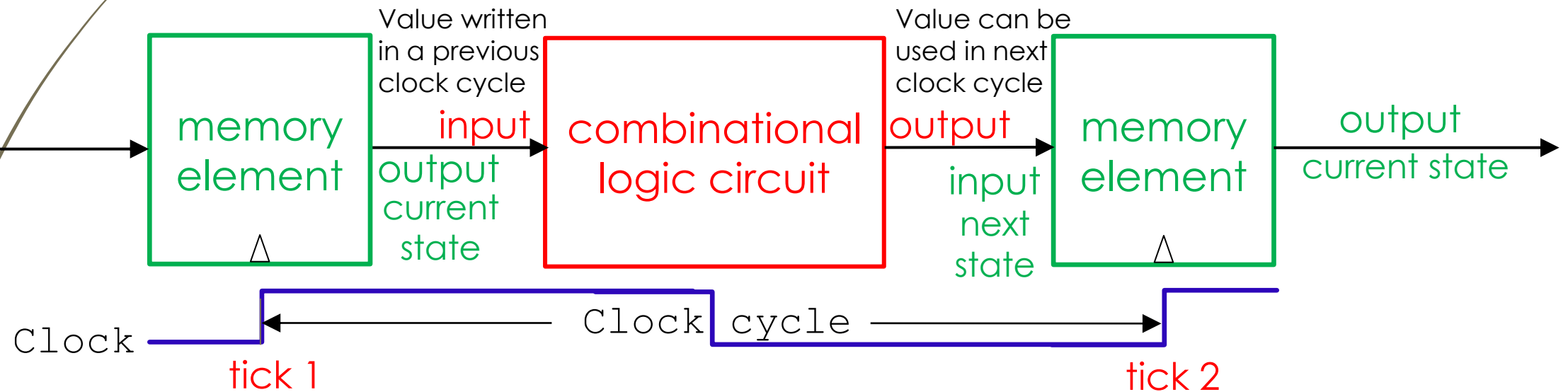


➤ Writing

- *Sequential logic circuit* -> memory elements
- Remembers only when clock “ticks” (rising edge of clock)

How to put them together:

- Combinational logic circuits execute an instruction
- Sequential logic circuit (memory elements) hold results (state), i.e., “remember”



Summary

- We have now started to explore how the **microprocessor executes machine code instructions** (series of 0's and 1's)
- Hardware of a microprocessor (datapath)
 - Made of **combinational logic circuits, sequential logic circuits, clock**
 - We put **combinational logic circuits** and **sequential logic circuits** together
 - > datapath of a microprocessor
- “Constructed” our first microprocessor: executes instructions sequentially

Next Lecture

- Instruction Set Architecture (ISA)
 - Definition of ISA
- Instruction Set design
 - Design principles
 - Look at an example of an instruction set: MIPS
 - Create our own
 - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
 - Execution of machine instructions (datapath)
 - Intro to logic design + Combinational logic + Sequential logic circuit
 - Sequential execution of machine instructions
 - Staged execution of machine instructions
 - Pipelined execution of machine instructions + Hazards